



FRONTEND DESIGN · 2026

Arquitetura em Camadas & HTML Fundamental

A separação entre conteúdo, estilo e comportamento — e os primeiros passos com HTML de verdade.

• Aula 03 · Semestre 1/2

Recapitulando a **Aula Anterior**

Na aula passada, exploramos o **VS Code** — o editor que vai nos acompanhar em toda a jornada. Aprendemos a navegar pela interface, usar atalhos essenciais, criar arquivos e aproveitar a inteligência do editor.

Agora que dominamos a **ferramenta**, é hora de entender a **matéria-prima**. Hoje vamos mergulhar na arquitetura que sustenta toda a web e escrever HTML de verdade.

Agenda da **Aula**

TEMPO	ATIVIDADE	FOCO
00-10 min	Revisão Relâmpago	Recapitular a aula anterior: VS Code, atalhos, arquivos. Conexão com o próximo passo.
10-25 min	Arquitetura em Camadas	A separação HTML / CSS / JS. Analogia da construção. Demonstração ao vivo.
25-40 min	O Papel de Cada Camada	Conteúdo, estilo e comportamento. O que acontece quando uma camada falha?
40-55 min	HTML Fundamental	Estrutura base: DOCTYPE, html, head, body. Tags estruturais essenciais.
55-70 min	Tags na Prática	Headings, parágrafos, listas, links, imagens. Atributos essenciais.
70-85 min	Semântica Inicial	Tags semânticas vs. divs genéricas. Por que semântica importa (SEO, acessibilidade).
85-100 min	Exercício Prático	Criar uma página HTML completa no VS Code com tags semânticas.

As Três **Camadas** da Web

Todo site é construído com três linguagens, e cada uma tem uma **responsabilidade clara e separada**. Essa separação se chama **Arquitetura em Camadas** — e é um dos princípios mais importantes do desenvolvimento web.

Pense em um prédio: você não mistura a fundação com a pintura, nem a fiação elétrica com o acabamento. Cada parte tem seu papel. Na web, a lógica é a mesma.



HTML — Conteúdo

A **estrutura** da página. Define *o que* existe: textos, imagens, links, listas. É o esqueleto — sem ele, não há página. Analogia: as **paredes, colunas e fundação** de um prédio.



CSS — Estilo

A **apresentação visual**. Define *como* tudo aparece: cores, fontes, espaçamentos, layout. Analogia: a **pintura, decoração e acabamento** do prédio.



JavaScript — Comportamento

A **interatividade**. Define *o que acontece* quando o usuário interage: cliques, animações, validações, requisições. Analogia: a **elétrica, hidráulica e automação** do prédio.

TUDO MISTURADO

HTML + CSS + JS inline

Estilos no atributo style, JavaScript no onclick, CSS dentro do HTML. Funciona — mas é impossível de manter, reutilizar ou escalar.

CAMADAS SEPARADAS

Cada arquivo com sua função

HTML em .html, CSS em .css, JS em .js. Cada camada pode ser editada, cacheada e testada independentemente. **Profissional e escalável.**

Os Benefícios da **Separação**

A separação de responsabilidades não é burocracia — é o que permite que projetos cresçam sem virar um caos. Cada benefício se multiplica conforme o projeto fica maior.



Manutenção

Cada camada pode ser modificada independentemente. Mudar o visual do site inteiro? Altere apenas o CSS — o HTML permanece intacto.



Trabalho em Equipe

Designer cuida do CSS, dev do JS, conteudista do HTML. A separação permite que times trabalhem em paralelo sem conflitos.



Resiliência

Se o CSS não carregar, o conteúdo ainda aparece. Se o JS falhar, o HTML e o estilo continuam funcionando. Cada camada é uma rede de segurança.



Performance

O navegador pode cachear CSS e JS separadamente. Mudou o conteúdo? Só o HTML é baixado de novo — os estilos e scripts continuam no cache.

E QUANDO UMA CAMADA FALHA?

Sem CSS

Conteúdo legível, mas sem estilo

O HTML puro é exibido com o estilo padrão do navegador. Feio, mas funcional. O conteúdo continua acessível e navegável.

Sem JS

Página estática, mas visível

Links funcionam, formulários enviam (com action), imagens aparecem. Apenas interações dinâmicas são perdidas.

Sem HTML

Nada existe

Sem HTML, não há página. CSS e JS não têm onde atuar. O HTML é a camada obrigatória — as outras são opcionais.

05 — ESQUELETO

Estrutura Base do HTML

Todo documento HTML segue a mesma estrutura fundamental. Esses quatro elementos formam o **esqueleto** de qualquer página web — sem eles, o navegador não sabe como interpretar o conteúdo.

<!DOCTYPE>

Declaração de Tipo

Informa ao navegador que o documento usa HTML5. Sempre na primeira linha. Sem ele, o browser entra em "quirks mode" e renderiza de forma imprevisível.

<html>

Elemento Raiz

Contêiner de toda a página. O atributo lang="pt-BR" define o idioma — essencial para leitores de tela e motores de busca.

<head>

Metadados

Informações sobre a página que não são exibidas diretamente: título, charset, viewport, links para CSS, meta tags para SEO e redes sociais.

<body>

Conteúdo Visível

Tudo que o usuário vê e interage fica aqui: textos, imagens, links, formulários. É o palco onde a página acontece.

```

<!-- Estrutura base de todo documento HTML -->
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Minha Página</title>
  </head>
  <body>
    <h1>Olá, mundo!</h1>
    <p>Meu primeiro HTML no VS Code.</p>
  </body>
</html>

```

Dica do VS Code: Digite ! e pressione Tab — o Emmet gera toda essa estrutura automaticamente. É o atalho mais útil do HTML.

TAGS ESSENCIAIS DO <HEAD>

T

<title>

O título que aparece na aba do navegador e nos resultados do Google. Deve ser descritivo e único para cada página.

<>

<meta charset>

`<meta charset="UTF-8">` — define a codificação de caracteres. UTF-8 suporta acentos, emojis e caracteres de qualquer idioma.



<meta viewport>

`<meta name="viewport" content="width=device-width, initial-scale=1.0">` — obrigatório para sites responsivos. Sem ele, mobile renderiza a página como se fosse desktop.



<link>

Conecta recursos externos à página. O mais comum: `<link rel="stylesheet" href="style.css">` para carregar o CSS externo.

Tags Essenciais do `<body>`

Essas são as tags que você vai usar em **praticamente toda página**. Dominar essas tags é dominar a base do HTML — todo o resto se constrói sobre elas.

T

`<h1>` a `<h6>`

Títulos hierárquicos. `<h1>` é o mais importante (um por página), `<h6>` o menos. Definem a **hierarquia do conteúdo** — não são para mudar tamanho de fonte.

≡

`<p>`

Parágrafo de texto. O elemento mais básico para conteúdo textual. Browsers adicionam margem vertical automaticamente entre parágrafos.

🔗

`<a>`

Âncora — cria links. O atributo `href` define o destino. Pode apontar para outra página, uma seção da mesma página (`#id`) ou um recurso externo.

🖼️

`<img>`

Insere imagens. Dois atributos obrigatórios: `src` (caminho da imagem) e `alt` (descrição textual para acessibilidade e SEO). Tag auto-fechante.

≡

`<ul>`, `<ol>`, `<li>`

`<ul>` = lista não ordenada (bullets). `<ol>` = lista ordenada (números). `<li>` = cada item da lista. Listas podem ser aninhadas.

T

`<strong>`, `<em>`

`<strong>` = importância forte (negrito). `<em>` = ênfase (itálico). São **semânticos** — não use `<b>` e `<i>` para significado.

←!— Exemplo com as tags essenciais →

```
<h1>Receita de Brigadeiro</h1>
```

```
<p>0 brigadeiro é o <strong>doce mais popular</strong> do Brasil.</p>
```

```
<h2>Ingredientes</h2>
```

```
<ul>
```

```
  <li>1 lata de leite condensado</li>
```

```
  <li>1 colher de manteiga</li>
```

```
  <li>3 colheres de achocolatado</li>
```

```
</ul>
```

```
<p>Veja mais em <a href="https://receitas.com">Receitas.com</a>.</p>
```

```

```

HTML Semântico

HTML semântico significa usar tags que **descrevem o significado** do conteúdo, não apenas sua aparência. Em vez de `<div>` para tudo, usamos tags que comunicam a **função** de cada bloco.

Um `<div>` não diz nada. Um `<nav>` diz "isso é navegação". Um `<article>` diz "isso é conteúdo independente". Essa diferença é **invisível para o olho**, mas fundamental para máquinas, leitores de tela e SEO.



`<header>`

Cabeçalho da página ou de uma seção. Normalmente contém o logo, navegação principal e título do site.



`<nav>`

Navegação principal. Agrupa links de navegação do site. Leitores de tela usam essa tag para pular direto ao menu.



`<main>`

Conteúdo principal da página. Deve ser único — apenas um `<main>` por página. Exclui header, footer e sidebars.



`<article>`

Conteúdo independente e autocontido. Um post de blog, uma notícia, um comentário — algo que faz sentido sozinho, fora do contexto da página.



`<section>`

Agrupa conteúdo tematicamente relacionado. Diferente de `<div>`, comunica que aquele bloco tem um significado temático.



`<footer>`

Rodapé da página ou de uma seção. Informações de contato, copyright, links secundários. Pode haver mais de um por página.

`<DIV>` VS. TAGS SEMÂNTICAS

`<div>`

Container Genérico

Não tem significado semântico. Use apenas quando nenhuma tag semântica se aplica — para agrupamento visual ou estilização.

`<section>`

Seção Temática

Agrupa conteúdo relacionado por tema. Deve ter um heading (h1-h6). O leitor de tela anuncia como "região".

`<article>`

Conteúdo Independente

Conteúdo que faz sentido sozinho. Se você pode republicar em outro contexto e continua fazendo sentido, é um article.

`<aside>`

Conteúdo Lateral

Informação tangencialmente relacionada: sidebar, pull quote, publicidade. Complementa o conteúdo principal sem ser essencial.

POR QUE SEMÂNTICA IMPORTA?



SEO

Motores de busca entendem a estrutura do conteúdo. Um `<nav>` é navegação, um `<article>` é conteúdo principal. `<div>` não diz nada ao Google.



Acessibilidade

Leitores de tela usam tags semânticas para navegar. Um usuário cego pode pular direto ao `<main>` ou listar todos os `<nav>` da página.



Legibilidade do Código

Outro desenvolvedor lê `<header>` e sabe imediatamente o que é. Com `<div class="header">`, precisa interpretar — mais lento, mais propenso a erros.

←!— ❌ Sem semântica — tudo é div →

```
<div class="header">...</div>
<div class="nav">...</div>
<div class="content">...</div>
<div class="footer">...</div>
```

←!— ✅ Com semântica — cada tag tem significado →

```
<header>...</header>
<nav>...</nav>
<main>...</main>
<footer>...</footer>
```

08 — PRÁTICA

Mão na Massa

Exercício: página de perfil pessoal

Agora que você conhece as tags essenciais e a semântica do HTML, vamos criar uma **página de perfil pessoal** usando tudo que aprendemos. Use o VS Code e aproveite o Emmet (! + Tab).

1. Crie um novo arquivo **perfil.html** no VS Code.
2. Use ! + Tab para gerar a estrutura base.
3. Altere o <title> para seu nome.
4. Dentro do <body>, crie a seguinte estrutura semântica:

```
<header>
  <h1>Seu Nome</h1>
  <p>Estudante de Engenharia de Software – FIAP 2026</p>
</header>

<nav>
  <a href="#sobre">Sobre</a>
  <a href="#hobbies">Hobbies</a>
  <a href="#contato">Contato</a>
</nav>

<main>
  <section id="sobre">
    <h2>Sobre Mim</h2>
    <p>Escreva um parágrafo sobre você... </p>
    
  </section>

  <section id="hobbies">
    <h2>Meus Hobbies</h2>
    <ul>
      <li>Hobby 1</li>
      <li>Hobby 2</li>
      <li>Hobby 3</li>
    </ul>
  </section>

  <section id="contato">
    <h2>Contato</h2>
    <p>Email: <a href="mailto:seu@email.com">seu@email.com</a></p>
  </section>
</main>

<footer>
  <p>&copy; 2026 – Seu Nome. Feito na FIAP.</p>
</footer>
```

5. Personalize com seu nome, seus hobbies e suas informações.
6. Salve e abra no navegador (clique direito → Open with Live Server, ou arraste o arquivo para o browser).
7. **Desafio extra:** adicione uma lista ordenada () com seus 3 filmes ou séries favoritos.

Lembre-se: A página vai parecer "sem estilo" — é assim mesmo! O HTML define apenas a **estrutura**. Na próxima aula, vamos adicionar CSS para transformar essa estrutura em algo visualmente bonito.